Northstar Payments: QA architecture

A concrete design for AI assistance across a release workflow with 75 offshore QA staff.

75

offshore QA staff

One payment release.
A platform the next team can reuse.

Technical SDs and leads • Fictional fintech • Proposed architecture

The assumed application landscape

Payment APIs offer the first controlled test surface. Other applications follow their own readiness.

Payment services
Java / Spring Boot
PostgreSQL journal
Versioned payment API

First pilot boundary

↔
↔
↔

Web + mobile

React journey; existing Selenium and Appium suites

Provider integration

WireMock during service tests; provider sandbox for integration

Settlement batch

Legacy Java + SQL; shared nightly environment and file reconciliation

Assumed stack: Java/Spring Boot, React, PostgreSQL and Azure. Existing automation remains useful.

The shared QA platform

The model proposes artifacts. Reviewers approve them. Existing runners produce evidence.

1. QA context
Stories, tests, defects
Versioned business rules

2. AI workbench
Draft and explain
Proposed artifacts only

3. Human review
Approve expected results
Review test-code changes

4. Test runners
Existing CI + frameworks
Pinned build and fixtures

5. Run evidence
Assertions, logs, traces
AI drafts failure summaries

6. Release review
QA + release owner
Defects and exceptions

Shared foundation: synthetic data · dependency stubs · artifact IDs · ownership · model configuration

Adapters, context assembly and evidence linking are integration work to build. No turnkey platform is implied.

PAY-142: the payment contract

CONDITION	EXPECTED BEHAVIOR
Same key + same payload	Return the same transfer ID and result. Post the journal once.
Same key + changed amount	Reject the conflicting request without another posting.
Concurrent retries / duplicate callbacks	Converge on one transfer and one balanced journal.
Expected final state	Payer CAD 900.00; recipient CAD 100.00; one journal with two balancing entries.

Authored business rules: initial balances CAD 1,000.00 and CAD 0.00; transfer CAD 100.00; no fees, FX or other activity. No regulatory claim.

The retry failure and its detection

The provider accepts a transfer, but its response times out. A retry reaches the posting path again.

Customer		Payment API		Provider		Customer retries
Send CAD 100	→	Key: pay-142	→	Accept; response times out	→	Reuse pay-142

Injected defect

2 journals

Payer CAD 800
Recipient CAD 200

FAIL · duplicate posting

Required behavior

1 journal

Payer CAD 900
Recipient CAD 100

PASS · expected state

Illustrative outcomes for initial balances of CAD 1,000 and CAD 0. A journal contains one debit and one credit. No fees, FX or other activity.

Injecting a duplicate-posting defect should fail the journal-count and balance assertions. A correct success toast cannot override that failure.

Requirements and test design

PAY-142 starts with an ambiguity: does a timeout mean failure, success or unknown?

Requirements

AI assistance

Draft ambiguity questions and identify affected payment journeys.

Output and owner

Versioned acceptance criteria and an open-question log. Product owner + domain QA.

Human check

Product owner confirms that a retry uses the same key and returns the original transfer.

Test design

AI assistance

Draft positive, boundary, retry and concurrency cases with requirement links.

Output and owner

Reviewed test matrix with independently defined expected results. Domain QA lead.

Human check

Include same-key/different-payload, parallel retry and duplicate callback cases.

Domain QA and the product owner approve expected behavior before generated tests can validate it.

Test data and environment preparation

Synthetic accounts, known balances and a controlled provider response make the retry repeatable.

Test data

AI assistance

Draft seed scripts and scenario combinations using synthetic records.

Output and owner

Versioned fixtures with isolated accounts and known starting balances. Test-data engineer.

Human check

Verify relationships and exact balances before running tests.

Environment

AI assistance

Summarize readiness failures and draft dependency stub mappings.

Output and owner

Reproducible test run with pinned build, data and stub versions. Platform engineer + QA.

Human check

A deterministic preflight checks health, schema and fixture readiness.

Container-based service tests do not replace real integration, mobile or batch testing. Validate stubs against the provider contract.

Generated code enters an ordinary review

Use the approved case matrix and repository examples to produce a candidate test change.

1 / AI CANDIDATE

A weak first draft

```
assert status == 200
```

A success response can hide two journals.

2 / HUMAN REVIEW

Reject and repair

Require the original transfer ID, one journal and independently agreed balances.

Unexplained skips cannot satisfy the case.

3 / CI CHALLENGE

Prove the assertion

Expected balances: **90000 / 10000** minor units.

The correct build must pass; the deliberate duplicate-posting variant must fail.

Authored PAY-142 review example; pseudocode and expected results are illustrative. Other required cases and real integration checks remain necessary.

The reviewer checks the oracle. A deliberate duplicate-posting fault must fail before the candidate joins the required suite.

The appropriate test layer

LAYER	EXAMPLE IMPLEMENTATION
Service and API	REST Assured + JUnit. Cover retry, concurrent requests, idempotency conflicts and journal outcomes.
Web journey	Playwright for the new React flow. Check pending state, retry action and the final customer message.
Existing browser and mobile	Keep owned Selenium and Appium coverage. Change frameworks only when maintenance evidence supports it.
Batch and integration	Retain settlement-file reconciliation and provider sandbox checks with explicit scheduling.

Test distribution follows behavior and observability. A new browser framework does not solve a shared settlement environment.

Execution retains required coverage

CI records the build, fixture, stub and test revisions before runners execute the pack.

Execution

AI assistance

Recommend affected tests and summarize run evidence.

Output and owner

Runner results, traces and mandatory-suite completion record. Application QA team.

Human check

Keep the mandatory payment suite while validating selection quality.

AI test-selection recommendations run in shadow mode first. Required payment checks continue on every eligible release.

Triage preserves the original test intent

A failed journal assertion belongs to the product defect path until evidence establishes otherwise.

Triage & retest

AI assistance

Cluster related failures and draft evidence-linked defect or repair proposals.

Output and owner

Reviewed defect disposition, fix and fresh rerun evidence. QA lead + developer.

Human check

Separate product defects, flaky tests and environment faults. Recheck original intent after a patch.

Capture failed tests, skipped tests and retries. A repair that removes an assertion does not count as a passing outcome.

A traceable release decision

Every generated artifact and run links back to the approved business behavior.

PAY-142
Approved behavior

Test revision
Reviewed assertions

Run manifest
Build, data, stub

Run evidence
Results + traces

Disposition
Defect / fix / rerun

Release owner
Recorded decision

> AI can draft the summary. Missing assertions or unresolved defects remain visible to the release reviewer.

An absent result remains unknown. The release owner resolves exceptions and the QA lead reviews the evidence pack.

What the demonstration proves

TRY IN THE CASE EXPLORER	EXPECTED OUTCOME
Normal transfer	One posting and expected balances: the modeled checks pass.
Retry with injected bug	Two postings and incorrect balances: the modeled checks block release.
Retry with deduplication	One posting after two attempts: the modeled checks pass.
Duplicate callback	Repeat the event with the same identity and inspect the journal outcome.

The browser demonstration is a deterministic teaching model. It does not execute REST Assured, call a payment service or prove production correctness.

Readiness across six dimensions

These dimensions describe the assumed delivery profile. Review the wider dependency register before approving a workflow.

DIMENSION	ASSUMED MIXED PROFILE	FIRST IMPLEMENTATION PRIORITY
Requirements	Versioned stories; some ambiguity	Confirm payment outcomes before generation
DevOps	CI available; deployment partly manual	Pin build and deployment revisions
Cloud & environments	Cloud test environment; data reset shared	Isolate accounts and script data reset
Application testability	Payment API testable; batch dependency shared	Pilot the API and control provider behavior
Automation	Java automation usable; UI pack noisy	Stabilize owned tests and retain required coverage
Offshore delivery	Domain expertise strong; automation skills uneven	Pair reviews and define the next owner

Assess each application separately. Unknown infrastructure, data, virtualization, AI access or ownership must be resolved before the affected workflow can proceed.

Offshore handoff and platform ownership

Each handoff carries a reproducible artifact and a named next owner.

Owner / artifact	Shared overlap	Offshore workday	Next overlap
Product + domain QA	Approve PAY-142 Rule + scenario matrix ↓	Questions wait for a named reviewer	Resolve ambiguity Updated rule → QA
Offshore QA + engineers	Accept the case Owner + review window →	Draft, review and run AI proposes; engineer approves ↓	Repair and retest Fresh run → release review
Platform + CI	Provide pinned fixtures Supported runner + reset →	Retain original evidence Run ID + failure + next owner →	Reproduce the failure Escalate missing evidence

Pilot team: one QA lead, four domain testers, two automation engineers and one data/environment engineer. Product, development and platform support contribute separately.

Effort includes review and operation

Record time at the eight workflow stages using the same boundary for baseline and assisted packs.

Purple segment = review. Total assisted effort = 222h.

78h
gross capacity
300 − 222

66h
after operation
78 − 12

33h
usable capacity
66 × 50%

15 packs
to recover 480h setup
rounded up, steady assumptions

Targets assume verified adoption prerequisites. Negative capacity remains possible; do not extrapolate one pack to all 75 staff or combine overlapping benefits.

Pilot validation and stop conditions

STEP	PROPOSED VALIDATION
Design	Baseline one current pack. Compare at least two assisted packs with similar scope and matched conventional tasks.
Record	Capture task complexity, reviewer time, corrections, failures, waits and environment incidents.
Stop or repair	Pause expansion for an escaped money-movement defect, unreliable fixtures or tests that pass the injected fault.
Expand	Require trustworthy mandatory coverage, positive net capacity and a successful second-squad reuse trial.

Small pilot samples support a delivery decision with uncertainty. They do not establish a general causal productivity rate.

Architecture questions for the leads

1. “Can we deploy a known build, reset the data and replay a provider timeout today?”
2. “Where do payment outcomes become observable: API, journal, callback or batch file?”
3. “Which framework already has maintainers and reliable CI execution?”
4. “Who owns reusable fixtures and who approves the expected financial behavior?”

Review the workflow map and a real failed release together. The first implementation backlog should follow those answers.

Platform prerequisites for the pilot

SCOPE FOLLOWS PROVEN CAPABILITY

Inputs + people + evidence + approved AI access + a measured baseline

DRAFTING

Reviewed scenarios

Approved rules and observable outcomes.

↓

Domain QA accepts the scenario matrix.

Assess test design →

DIAGNOSIS ONLY

Evidence-linked triage

Retained failures, known dispositions and usable diagnostics.

↓

A reviewer checks the diagnosis; no retest runs.

Assess diagnosis →

EXECUTION

Reviewed test changes

Add repeatable environments, CI, fixtures, provider control and reliable runners.

↓

Independent assertions challenge the candidate.

Assess automation →

Unknown evidence remains a gap. Platform owners, reviewers and supplier leads must accept named actions and review dates before adoption.

Service virtualization and real integration

Control the payment provider for repeatable tests; keep a separate real-integration gate.

Reviewed payment test
Same key · retry · expected balances

→

Payment service
Pinned build and isolated fixture

→

Controlled provider
Timeout · delay · duplicate callback

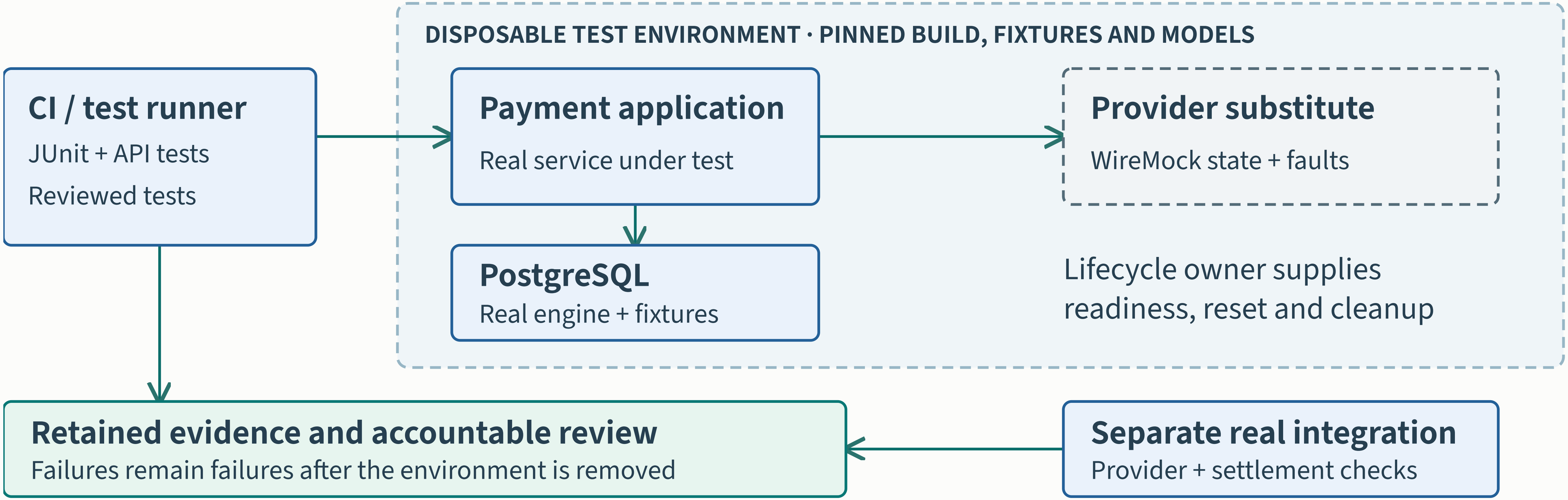
Separate integration gate

Versioned contract + provider sandbox / real batch validation ↔ Refresh the substitute when the real dependency changes

Proposed payment test architecture. A passing substitute is evidence about the modeled behavior, not proof of compatibility with every real dependency. Dependency strategy and sources →

WireMock is an HTTP example. Contract fidelity, callback behavior and batch access need explicit owners; passing a stub does not prove end-to-end correctness.

Northstar Payments: test environment



The CI runner checks the real payment service and database against reviewed provider models. Retain separate provider and settlement evidence.

Technology roles in the payment pilot

RUN

Containerization

Package and run real software with repeatable dependencies.

EXAMPLE

A PostgreSQL instance created for one test run.

SIMULATE

Service virtualization

Replace selected dependency behavior with a controlled model.

EXAMPLE

A WireMock provider with an intentional timeout.

CHECK

Contract testing

Check whether consumer and provider interactions agree.

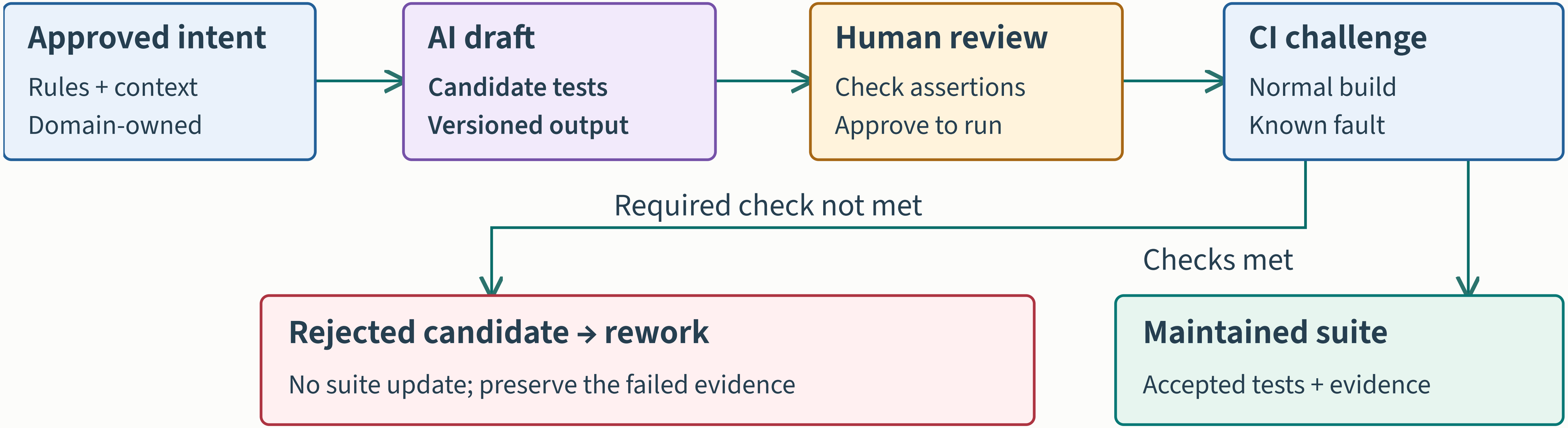
EXAMPLE

A provider change fails a reviewed Pact contract.

These capabilities complement each other. They do not establish complete business correctness or replace all real integration checks. Technology choices and limitations

Choose Testcontainers, Compose or an existing supported environment for the test scope. Mobile devices and legacy settlement retain their own integration requirements.

A candidate test must earn acceptance



PAY-142 must catch an incorrect transfer, journal or balance. The guided example rejects the candidate and leaves the maintained suite unchanged.

A fair comparison for the payments pilot

Shared conditions · Comparable tasks, same framework, environment and acceptance rules

Conventional work

- Engineer drafts
- ↓ Reviewer checks
- ↓ Tests challenge behavior
- ↓ Retain results and effort

AI-assisted work

- AI + engineer draft
- ↓ Reviewer checks
- ↓ Tests challenge behavior
- ↓ Retain results and effort

Compare accepted outcomes · Include unsuccessful attempts, review, rework and operating costs

Proposed evaluation design informed by ANZ's study limitations and METR's selection findings. Separate modernization gains from incremental AI effects.

Use comparable tasks inside the proposed pilot. Include failures and review effort; retain the existing mandatory payment checks.

The runtime from setup to cleanup

Proposed execution lifecycle. Numbered steps show order, not elapsed time.

1 **Create and check**
Pin the build, tests and configuration. Provision supported resources and wait for health checks.

2 **Seed and configure**
Create isolated synthetic accounts. Load the owned, versioned virtual-provider behavior.

3 **Execute and challenge**
Run required tests. The deliberate duplicate-posting variant must fail its independent assertion.

4 **Retain and review**
Keep original assertions, setup diagnostics and run IDs outside disposable resources. Review omissions.

5 **Clean up on every path**
Attempt teardown after success or failure. Escalate failed cleanup to the named platform owner.

6 **Reconcile integration**
Check the real provider and settlement separately. Contract drift triggers model review and rerun.

> Evidence survives cleanup. A missing or failed required check holds the release.

Repeatable execution is a platform capability. Retain failure evidence and attempt cleanup on every path; real integration remains separate.

Connect the tools behind the platform

1 / ASSUMED EXISTING TOOLS
Approved QA context
Azure Boards / Test Plans + repository
Product + domain QA
Story and test revisions → 2

2 / BUILD · INTEGRATION SERVICE
Context assembly
Versioned templates + context adapter
QA platform owner
Prompt + source IDs → 3

3 / VALIDATE ACCESS · CONFIGURE
AI workbench
Approved assistant / model endpoint
AI platform owner
Candidate test + provenance → 4

4 / ASSUMED REVIEW · EXTEND CHECKS
Human review
Domain check + reviewed pull request
Domain QA + automation engineer
Approved test commit → 5

5 / ASSUMED TOOLS · INTEGRATE
CI and test runners
Azure Pipelines · JUnit / REST Assured · Playwright
DevOps + application engineers
Assertions + run manifest → 6

6 / BUILD · JOINED EVIDENCE
Evidence catalog
Pipeline artifacts + Test Plans links
QA + release owner
Accountable release review

Runtime: CI runs approved commits in the isolated test environment. Real provider, settlement and mobile checks retain separate lanes.

Proposed integration. Validate access; fund context adapters and joined evidence.